Project Proposal: The Computer Pianist

A High-Speed, Reactionary Robot for Rhythm Game Automation
Physics 124

Dylan Dahlke and Ryan Moore

Feb 10, 2026

1 Motivation and Overview

1.1 The Limitation of Human Reflexes

The motivation for this project lies in the physical and biological limitations of human reaction time compared to the processing speed of modern microcontrollers. In a typical rhythm game environment, such as the mobile game *Piano Tiles 2*, a player is tasked with identifying a visual stimulus (a black tile appearing on a white background) and reacting by physically tapping the screen.

For a human, this process involves a complex chain of biological events: optical transduction in the retina, signal propagation through the optic nerve, neural processing in the visual cortex, decision-making in the frontal lobe, and finally, the propagation of motor signals to the muscles in the fingers. The average human reaction time to a visual stimulus is approximately 250 milliseconds. While this is sufficient for casual gameplay, it imposes a hard "speed limit" on performance. Physically, this limits a human player to a maximum frequency of approximately 4–5 Hz (notes per second). As game difficulty increases, note density often exceeds this biological bandwidth, making perfect performance physiologically impossible.

1.2 The Robotic Advantage

This project seeks to overcome these biological constraints by replacing the human neural pathway with an electronic circuit and software logic. By utilizing a microprocessor (Arduino Uno) operating at a clock speed of 16 MHz, the "reaction time" can be reduced from hundreds of milliseconds to mere microseconds. Unlike human nerve impulses, which travel at approximately 100 m/s, the internal signals of the microcontroller propagate at a significant fraction of the speed of light. This project explores the physics of optoelectronics and electromechanics to create a closed-loop system capable of playing *Piano Tiles* at speeds that are strictly impossible for a human player, limited only by the physical articulation time of the actuators.

1.3 Project Concept

The "Computer Pianist" is a non-invasive robotic apparatus designed to play rhythm games on a capacitive touch-screen device (smartphone or tablet). Unlike software-based "hacks" that manipulate the game's code directly, this project interacts with the device purely through physical means, mimicking a human player. The system consists of a structural "bridge" that spans the width of the mobile device. This bridge houses an array of four optical sensors (the "eyes") and four electromechanical actuators (the "fingers").

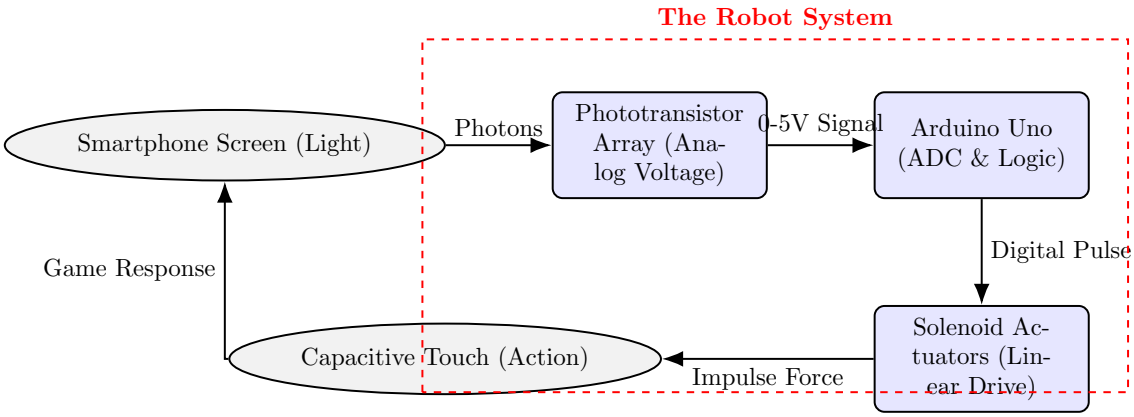

Figure 1: **System Data Flow:** A closed-loop feedback control system. The robot senses light (stimulus), processes the signal via the Arduino ADC, and triggers high-speed **Solenoid Actuators** to physically strike the screen (response).

1.4 Physics and Engineering Goals

The primary goal is to demonstrate the successful integration of analog sensing with digital control logic. We aim to construct a device that can perform three functions:

Sense: Observe the environment and mathematically describe it. By detecting the high-contrast transition between the white background and the black game tiles using phototransistors, our project quantizies the changes in light.

Process: Using a microcontroller to interpret these analog voltage changes through the phototransistors, logic states (Note Present vs. Note Absent) can be used to calculate the necessary kinematic timing. Translating analog input to an appropriate response is brain behind our project.

Actuate: Managing a physical response to the change in environment is critical in the

case of fast-moving piano tiles. Our process shall trigger a linear solenoid to physically depress the screen surface using a conductive tip, simulating a finger touch.

This project allows us to apply the skills learned in the first weeks of Physics 124specifically analog inputs, PWM output, and logic thresholdsto a real-world application involving feedback loops, hardware interrupts, and precise latency management.

2 Functional Definition

This section details the operational logic of the Computer Pianist, defining the system's reactions to all foreseeable operating states and stimuli using a Finite State Machine (FSM) architecture.

2.1 The Operational Environment

The device functions by placing a smartphone running a 4-lane rhythm game (e.g., *Piano Tiles*) underneath a stationary sensor/actuator bridge. The game presents a scrolling display where black tiles move from the top of the screen to the bottom at velocity v. The "hit line" (where the solenoid plunger contacts the screen) is located at a fixed distance Δy downstream from the optical sensors.

2.2 Initialization and Calibration State

Upon receiving power, the Computer Pianist enters an **Initialization State**. Because we have transitioned to Hardware Comparators for zero-latency detection, the threshold definition of "Black" vs. "White" is no longer calculated by the Arduino, but set physically via potentiometers.

The Arduino executes a `setup()` routine to verify the hardware readiness:

1. **Safety Lock:** The system immediately writes all Solenoid Driver pins LOW to prevent accidental actuation during startup.

2. **Logic Verification:** The system reads the four Digital Input pins from the comparators. It assumes the game is on the "Start" screen (All White). If any pin reads LOW (detecting "Black"), it implies a sensor error or miscalibration. The system will hold in this state until all sensors read HIGH.

3. **Confirmation:** Once the sensors are verified as "Idle," a dedicated status LED blinks

three times to indicate that the calibration is valid and the system is transitioning to the Active State.

S

2.3 The Active Sensing Loop

Once calibrated, the system enters the main `loop()`. However, unlike a standard polling loop, the "sensing" is handled asynchronously by the hardware.

Stimulus: A black tile scrolls down the screen.

Hardware Detection: As the leading edge of the tile passes underneath the phototransistor, the reflected light intensity decreases, causing the sensor voltage to drop. This analog voltage is constantly compared against the reference voltage (V_{ref}) by the external LM339 Comparator.

Interrupt Trigger: When the sensor voltage dips below V_{ref} (Black Tile), the Comparator output snaps from HIGH to LOW. This falling edge triggers the Arduino's Pin Change Interrupt (PCINT), instantly pausing the main loop to execute the actuation logic defined in the ISR.

2.4 The Actuation Sequence

When Condition B is met, the FSM transitions to the **Actuation State** for that specific lane. This is achieved by dedicating four digital pins on the Arduino board, one for each lane. This sets off three general actions to allow our phototransistors to properly translate the voltage change into a "press". To maximize speed, this sequence operates on a "Dead Reckoning" (Open-Loop) principle: we assume the high-speed solenoid will strike the target at the calculated time without waiting for force-sensor confirmation.

1. **Latency Delay:** Because the sensors are placed physically upstream from the actuators to prevent occlusion, the robot "sees" the note early. The software initiates a timer to wait for the duration Δt required for the tile to travel to the hit zone, governed by the kinematics equation (where v_{scroll} is derived from the pre-game calibration routine):

$$\Delta t = \frac{\Delta y}{v_{scroll}} \tag{1}$$

2. **Strike:** Once Δt elapses, the Arduino sends a digital HIGH pulse to the solenoid driver. The solenoid extends linearly to impact the screen, eliminating the rotational inertia delay associated with servo motors.

3. **Retract:** The solenoid is held active for a minimum duration ($t_{hold} \approx 20$ms) to ensure the capacitive screen registers the touch, then the signal is cut to allow the return spring to retract the plunger.

2.5 Handling Operating States and Errors

Signal Hysteresis (Debouncing): If the sensor detects noise or a screen scratch, it might trigger false positives. The logic includes a "Cooldown Timer" that prevents a single actuator from firing twice within a 50ms window (Subject to empirical testing during the high-speed phase).

Game Over State: If the robot misses a tile, the game freezes. The sensors see a static image, and the code inherently treats this as "no change," keeping the actuators idle. No dedicated "kill switch" is required.

Ambient Light Shift: Shadows falling across the phone may trigger false detections. We mitigate this mechanically by shrouding the sensors rather than increasing software complexity in the first iteration.

Phototransistor Overlap: Without a proper structure, it may be true that multiple phototransistors detect the same tile. To mitigate this, we shall construct a physical structure to block interference from external lanes.

2.6 Theoretical Latency Budget Analysis

To justify the necessity of the "Look Ahead" predictive algorithm, we must characterize the total system latency (T_{sys}). In a reactive system, the total time from stimulus to action is the sum of component delays:

$$T_{sys} = T_{rise} + T_{logic} + T_{mech} \tag{2}$$

The phototransistor has a finite response time to a change in photon flux, typically $\approx 15\mu s$. Previously, relying on the Arduino ADC introduced a sampling delay of $T_{ADC} \approx 104\mu s$. Following the feedback to optimize for speed, we have replaced ADC polling with an interrupt-driven architecture using analog comparators. This replaces the slow conversion time with a negligible comparator propagation delay and Interrupt Service Routine (ISR) latency ($T_{ISR} \approx 5\mu s$), reducing electronic latency by over 95%.

Furthermore, to address the articulation limits of servo motors, we have switched to high-speed solenoid actuators. While significantly faster than the servo's travel time ($25ms$), the solenoid plunger still possesses inertia and inductance that create a mechanical delay ($T_{mech} \approx 10 - 15ms$). Since T_{mech} remains orders of magnitude larger than the electronic delays, it is the dominant term. The "Look Ahead" variable Δt allows us to effectively negate this physical delay by firing the solenoid *before* the note reaches the hit line.

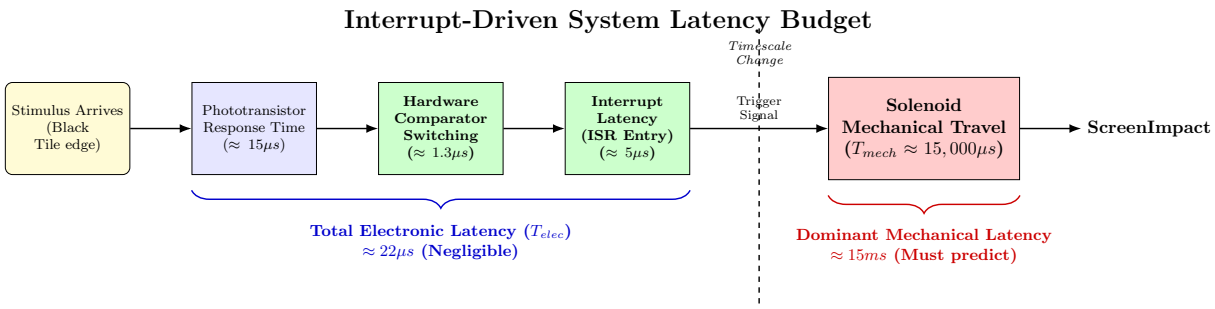

Figure 2: **System Latency Budget (Interrupt Architecture):** A timeline comparison of signal propagation delays. By switching from ADC polling to hardware comparators and interrupts, electronic latency (T_{elec}) is reduced to negligible microseconds. The mechanical transit time of the solenoid ($T_{mech} \approx 15ms$) now completely dominates the system, necessitating the predictive "Look Ahead" algorithm.

3 Sensors

3.1 Theoretical Basis of Sensing: Optoelectronic Physics

To understand the operation of the "eyes" of the Computer Pianist, we must examine the solid-state physics governing the chosen phototransistors. We are utilizing silicon-based NPN phototransistors which rely on the internal photoelectric effect to modulate current flow.

3.1.1 The Band Gap Mechanism

The core mechanism of detection is the generation of electron-hole pairs within the semiconductor material. Silicon has a band gap energy (E_g) of approximately 1.12 eV at room temperature. When a photon from the smartphone screen strikes the exposed Base-Collector junction of the phototransistor, it is absorbed if its energy (E_{ph}) exceeds this band gap:

$$E_{ph} = h\nu = \frac{hc}{\lambda} \geq 1.12 \text{ eV} \tag{3}$$

Where h is Planck's constant, c is the speed of light, and λ is the wavelength. The white light emitted by the smartphone LED backlight is a composite spectrum rich in peaks at 460nm (Blue) and broad phosphorescent emission from 500nm–700nm. These wavelengths correspond to photon energies well above 1.12 eV, ensuring efficient absorption and the promotion of electrons from the valence band to the conduction band.

3.1.2 Transistor Amplification (β)

A key design decision was choosing a phototransistor over a simple photodiode. While a photodiode generates a small photocurrent (I_{photo}) directly proportional to the incident light, this current is typically in the range of microamperes (μA), which is difficult to detect without a dedicated Op-Amp pre-amplifier stage.

The phototransistor solves this by using the Bipolar Junction Transistor (BJT) architecture for intrinsic amplification. The photon-generated carriers in the Base-Collector depletion region act effectively as a "base current" (I_b). In the active region of operation, the transistor amplifies this current by its forward current gain factor, β (or h_{FE}), which is typically between 100 and 1000 for standard devices. The resulting collector current (I_c) is given by:

$$I_c \approx \beta \cdot I_{photo} \tag{4}$$

This internal gain allows us to drive the Comparator inputs directly without complex amplification circuitry, simplifying our electrical footprint.

3.1.3 Voltage Divider Transfer Function

To convert this variable current (I_c) into a voltage (V_{out}) usable by the hardware interrupts, we place the phototransistor in a Common-Emitter voltage divider configuration.

We connect the Emitter to Ground and the Collector to the $+5V$ rail via the fixed resistor. The node between the Collector and the resistor is our output V_{out}. This output is fed directly to the inputs of the LM339 Analog Comparator. According to Kirchhoff's laws:

$$V_{out} = V_{cc} - (I_c \cdot R_{fixed}) \tag{5}$$

Substituting the optical relationship, we see the transfer function:

$$V_{out} = 5\text{V} - (\beta \cdot I_{photo} \cdot R_{fixed}) \tag{6}$$

This equation dictates the switching logic for our interrupts:

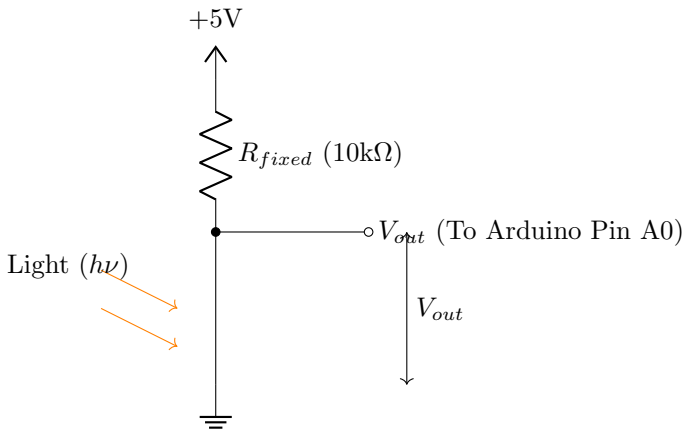

Figure 3: **Optical Sensor Circuit:** The NPN phototransistor in a common-emitter voltage divider. Note the Inverse Logic: As light intensity increases (White), Current (I_{photo}) increases, pulling V_{out} LOW. When a Black Tile arrives (Dark), Current drops, allowing V_{out} to be pulled HIGH ($5V$).

- **High Light (White Background):** I_{photo} is high. The voltage drop across R_{fixed} is large. V_{out} is pulled Low ($< V_{ref}$), keeping the Comparator Output LOW.
- **Low Light (Black Tile):** I_{photo} approaches zero (dark current). The voltage drop across R_{fixed} minimizes. V_{out} is pulled High ($> V_{ref}$), causing the Comparator to switch

HIGH and trigger the Interrupt.

3.2 Sensing the Physical touch with Force Sensors

No Pianist is complete without a good set of fingers. After sensing changes in light, we aim to understand the location of each finger (on or off the screen) and respond appropriately. To do so, force sensors will be applied to each finger tip.

Dead Reckoning vs. Closed Loop: While we primarily rely on Dead Reckoning for speed (as detailed in Section 2.4), we can also implement force sensors to test this assumption. Ideally, the force sensors will provide "Closed Loop" confirmation that a hit occurred. If we find that the sensors introduce too much latency, we will revert to pure Dead Reckoning and utilize the force sensors strictly for static calibration and "Game Over" state detection.

3.2.1 Force Sensor in Circuit

Based on the force sensor provided in the shopping list, we will be using a FSR 400 Series Round Face Resistor. Below is an diagram showing how we will use the force sensor.

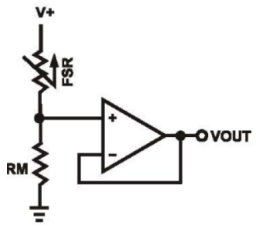

Figure 4: Diagram of our use of each FSR 400 Series Round Face Resistor

We will supply a constant Voltage across the Force Sensor and record the value for V_{out} and V_{in} using data cables hooked up to our Arduino board. We will also include an op-Amp to ensure we don't overload our arduino with current and it provides a simple voltage divider set-up allowing us to determine the resistance across the force sensor at any time. We will attach one data cable before force sensor and another at the end of the op-Amp.

3.2.2 Resistance

Based on the schematics for the part, the resistance without any pressure will be upwards of 10MΩ. When force is applied to the force sensor, the actuated resistance will be less than the non-actuated resistance, leading to a rise in voltage at V_{out}.

3.2.3 To the Arduino

By taking a measurement V_{out} at this non-actuated state, we record the lowest voltage that will be recorded at V_{out}. In our code, we can determine the state of each finger by comparing V_{out} to a non-actuated V_{out}. We should expect V_{out} to be greater at a "pressed" state. There will likely be some noise with our measurements and force sensing. We must account for and determine a range based on experimental testing.

3.3 Electronic Implementation

Implementing the theoretical model described above, we will construct the physical voltage dividers using 10kΩ fixed resistors.

- **Circuit Topology:** Each phototransistor is placed in series with a fixed resistor (nominally 10kΩ) between the +5V rail and Ground. The junction between the phototransistor and the resistor is connected to the Non-Inverting Input of the Comparator.
- **Operation:** As the light intensity reflected from the screen changes (White Background → Black Tile), the effective resistance of the phototransistor changes. This shifts the voltage at the junction point.
- **Readout:** The voltage is fed into an **LM339 Quad Comparator**. Instead of relying on the Arduino's slow ADC polling ($\approx 100\mu s$), the comparator provides a hardware-level digital transition in $< 1\mu s$ when the voltage crosses the threshold set by the tuning potentiometers.

4 Electrical Considerations

4.1 Microcontroller Interface

The central processing unit will be an Arduino Uno R3.

- **Inputs:** As detailed in the Sensors section, pins A0 through A3 are dedicated to the four sensor inputs.
- **Outputs:** We require four Digital I/O pins to control the solenoid drivers. We will utilize Digital Pins 3, 5, 6, and 9. Unlike servos which require PWM, these pins will simply toggle HIGH/LOW to energize the MOSFET gates. We will explicitly avoid using Digital Pins 0 and 1, as these are reserved for Serial communication.

4.2 Power Distribution and Isolation

A critical electrical consideration is the power draw of the actuators. Solenoids are high-current inductive loads. A standard push-pull solenoid can draw current spikes in excess of 1A when initially energized, far exceeding the limit of the Arduino's I/O pins (40mA).

- **The Issue:** The Arduino Uno's on-board 5V regulator cannot supply the high current required for four simultaneous coils. Furthermore, connecting a solenoid directly to a microcontroller would destroy the chip due to inductive voltage spikes (back-EMF).
- **The Solution:** We will implement a split power rail system with a dedicated driver stage.
  - **Logic Power:** The Arduino will be powered via the USB connection to a laptop.
  - **Actuator Power:** The four solenoids will be powered by a **Benchtop DC Power Supply** set to 6.0V.
  - **Driver Circuit:** As shown in Figure 5, we will use an N-Channel MOSFET to act as a high-current switch. A "Flyback Diode" (1N4001) is placed in parallel

with the solenoid to safely dissipate the magnetic energy when the field collapses, protecting the rest of the circuit.

– **Common Ground:** The negative terminal of the power supply is tied to the Arduino GND to provide a reference for the MOSFET gate signal.

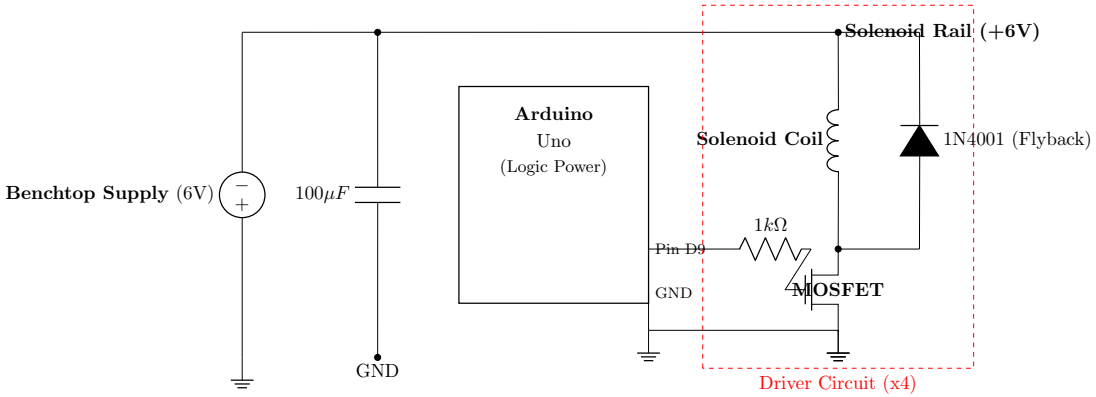

Figure 5: **Solenoid Driver Schematic:** The high-current Solenoid Rail (6V) is isolated from the logic. The Arduino toggles a MOSFET to complete the circuit. A flyback diode protects against inductive voltage spikes.

4.3 Signal Integrity and Noise Suppression

While the split power rail protects the microcontroller, the inductive nature of the solenoid coils introduces a secondary risk: **Electromagnetic Interference (EMI)**.

When a solenoid de-energizes, the collapsing magnetic field generates a high-voltage spike (back-EMF). To mitigate this:

- **Flyback Diodes:** We will place a 1N4001 diode in parallel with each solenoid coil (Cathode to +V) to safely dissipate the back-EMF current, protecting the MOSFETs.
- **Bulk Capacitance:** A large electrolytic capacitor (100μF) will be placed across the power rails to smooth out voltage dips caused by the sudden current demand of the solenoids firing.

5 Mechanical Considerations

5.1 Detailed Mechanical Design: The Sensor-Actuator Bridge

5.1.1 Structural Requirements and Physics of Stability

The primary mechanical constraint of the Computer Pianist is rigidity relative to the optical target. The optical sensors have a viewing angle of approximately 20°. At a height of 20mm above the screen, the "spot size" seen by the sensor is roughly 7mm in diameter. The game tiles are approximately 15mm wide. This implies that if the bridge structure vibrates or shifts by more than ±4mm laterally during operation, the sensors may lose alignment with the game lanes, resulting in false negatives. The actuation of the solenoids creates a sharp vertical mechanical impulse. A solenoid plunger snapping down in $< 15ms$ produces a distinct recoil. If the bridge mass is too low, these impulses will excite resonant frequencies, causing the "eyes" to jitter.

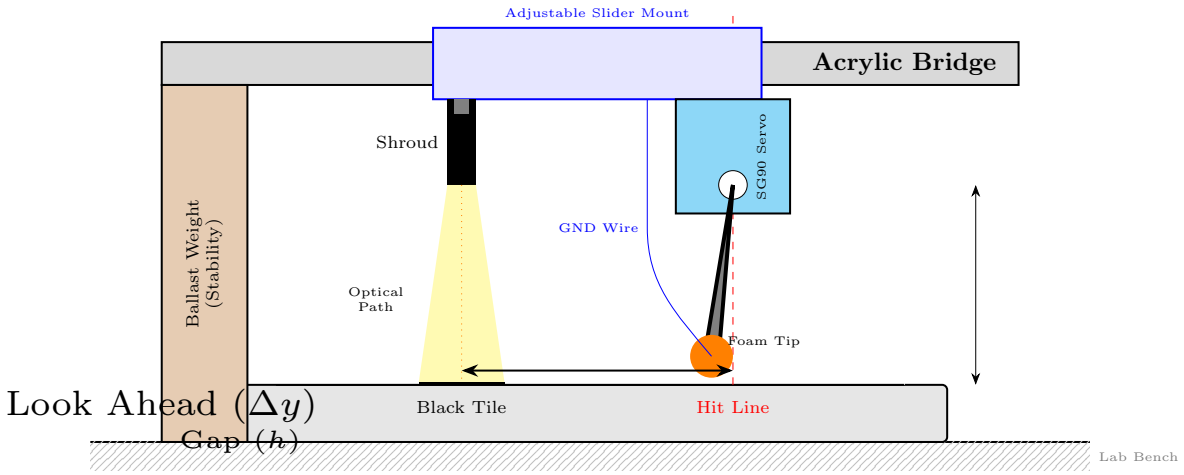

Figure 6: **Mechanical Side Elevation:** The assembly features a rigid acrylic bridge spanning the phone. Key elements include the "Look Ahead" distance (Δy) for latency compensation and the adjustable slider mount for aligning with variable lane widths.

- **Ballast Integration:** We will implement weighted base supports to increase the inertia of the base ($I = \sum mr^2$), making it resistant to the vertical recoil of the actuators.

- **Friction Maximization:** To prevent the entire unit from "walking" across the table due to vibration, the bottom surface will have a mat to prevent slippage.

5.1.2 Construction Methodology

We will utilize the physics machine shop's band saw and drill press for the primary fabrication, adhering to the safety protocols outlined in the class guide.

- **The Main Span:** We will cut a strip of 0.25-inch acrylic/wood sheet measuring 25cm x 4cm.

- **Sensor Shrouding:** To further isolate the optical path, we will fabricate "blinders" for the phototransistors. We will encase the sides of the phototransistors, leaving only the tip exposed. This effectively creates a "tunnel" for the light, reducing the acceptance angle and preventing adjacent white lanes from triggering a false "black" detection on a neighboring sensor.

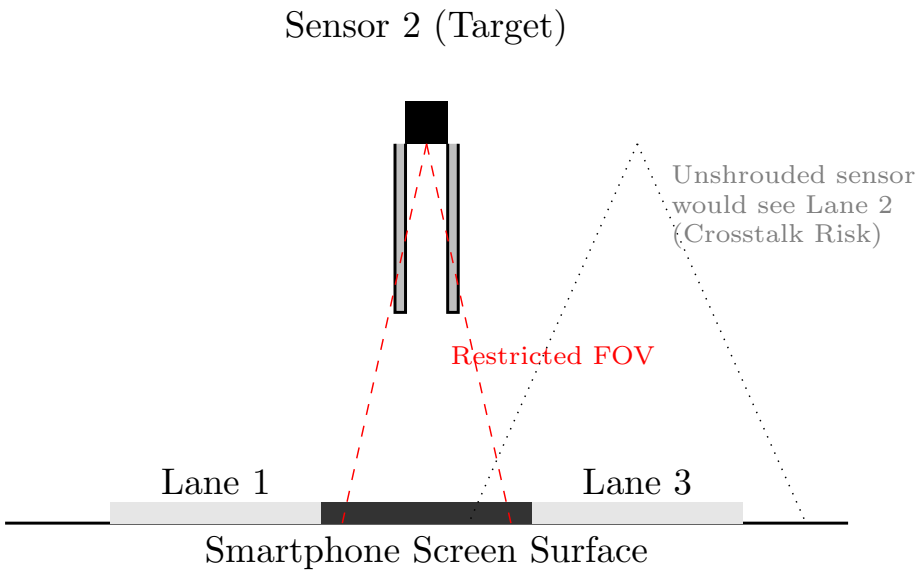

Figure 7: **Optical Isolation:** Without shrouding, the sensor's wide field of view (dotted lines) would detect tiles in adjacent lanes, causing false triggers. The opaque shroud restricts detection strictly to the target lane (red dashed lines).

5.2 The Actuators

5.2.1 Electromechanical Theory: Solenoid Dynamics

To achieve reaction times faster than the human hand, we have replaced the servo motors with **Linear Solenoid Actuators**. Unlike a servo which requires a complex Pulse Position Modulation (PPM) signal to set an angle, a solenoid is a binary device governed by the principles of electromagnetism.

Physics of Actuation (Lorentz Force)

The actuator consists of a wound copper coil and a ferromagnetic steel plunger. When current (I) flows through the coil, it generates a magnetic field (B) according to Ampere's Law. This field induces a magnetic flux that seeks to minimize the reluctance of the optical path, pulling the plunger into the coil center with a force (F) described largely by:

$$F \propto \frac{(N \cdot I)^2}{g^2} \tag{7}$$

Where N is the number of turns, I is the current, and g is the air gap distance.

- **Digital Control:** Since the device is binary, we discard the `Servo.h` library. Instead, we drive the solenoid using a **MOSFET Transistor** controlled by a simple Digital High/Low signal from the Arduino.
- **Speed Advantage:** Without the need to decode a PWM signal or drive a gear train, the solenoid plunger accelerates instantly upon energization, reducing the mechanical latency (T_{mech}) from $\approx 25ms$ (Servo) to $\approx 10 - 15ms$ (Solenoid).

Feasibility Analysis:

Solenoids typically exert maximum force when the air gap (g) is small. To ensure the "finger" has enough force to tap the screen from a hover height of 2mm, we will overdrive the coil with 6V (within thermal limits for short bursts). The lack of gears means there is zero backlash, providing a cleaner, sharper impact event compared to the plastic gears of an SG90.

5.2.2 Actuator Implementation

Based on the feasibility analysis, we will implement the actuation system using four Open-Frame Linear Solenoids.

- **Physical Mounting:** The solenoids will be mounted vertically to the main acrylic span using a custom 3D-printed bracket. This orients the plunger perpendicular to the screen surface, allowing for direct linear action without complex linkages.
- **Stroke Calibration:** Unlike servos which must be calibrated to specific angles, solenoids are binary. We will adjust the vertical height of the bridge such that the

"Hover" state (Plunger Retracted) holds the tip 2mm above the glass, and the "Strike" state (Plunger Extended) allows the foam to compress slightly against the screen.

- **Capacitive Coupling (The "Touch" Mechanism):** Standard steel plungers are electrically conductive but may not trigger the screen without a larger surface area.

    1. We will affix a block of conductive anti-static foam (recycled from IC packaging) to each plunger tip to increase the contact area and cushion the impact.
    2. Since the plunger is metal, we can ground the solenoid casing directly to the system's common ground rail. This turns the entire actuator assembly into a low-impedance capacitive probe, ensuring that the smartphone's touch controller detects a valid input upon contact.

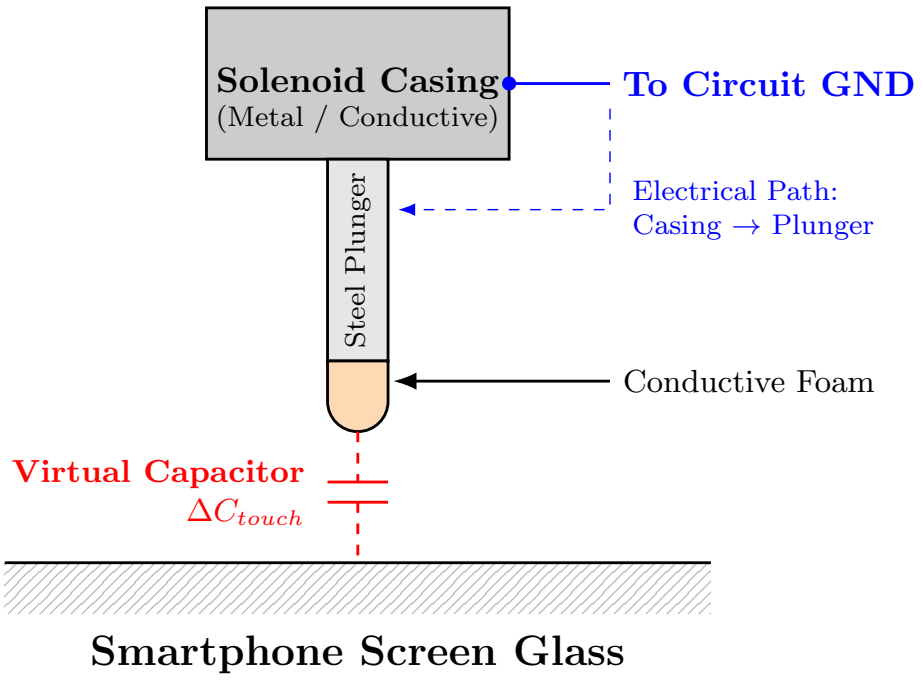

Figure 8: **Capacitive Touch Mechanism (Solenoid):** Unlike the previous servo design, the solenoid plunger is solid steel (conductive). By grounding the solenoid casing to the circuit's common ground, the entire assembly becomes an extension of the ground plane. The conductive foam tip completes the capacitive circuit (ΔC_{touch}) upon impact.

5.2.3 Custom 3D Printed Fingers

We have designed custom "finger" extensions using SolidWorks (Figure 9).

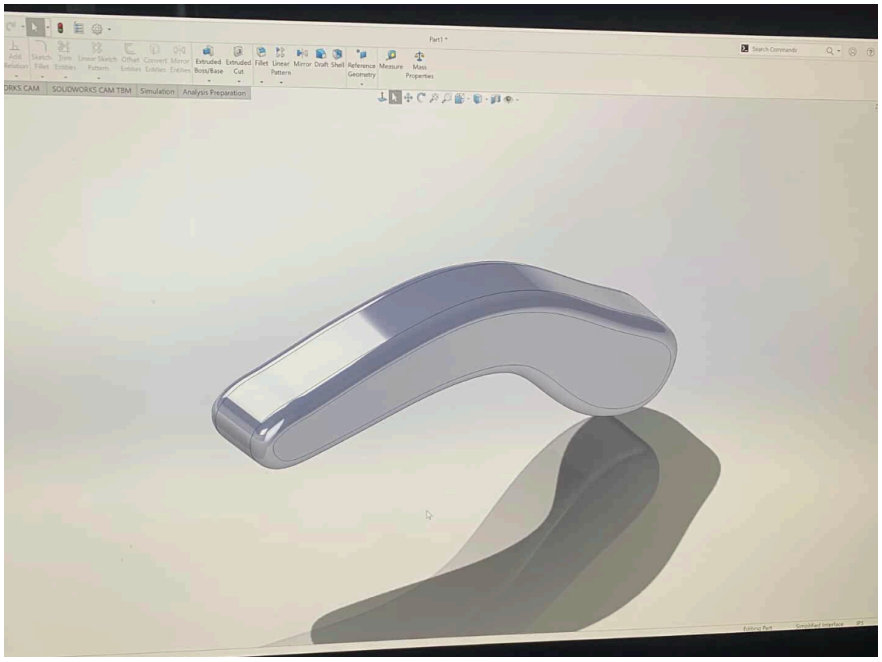

Figure 9: **CAD Model of Actuator Finger:** The custom extension features a curvature to clear the edge of the smartphone and an edge tip sized specifically to receive the conductive foam pad.

- **Fabrication:** The fingers will be 3D printed with material offering a high stiffness-to-weight ratio, ensuring the fingers are rigid enough to transmit force without adding excessive mass to the solenoid plunger, which would slow down the retraction speed.
- **Assembly:** The high-density conductive foam will be adhered to the flat tip of the printed finger. The grounding wire will be routed along the spine of the finger and inserted into the foam, completing the capacitive circuit to the common ground rail.

6 Interfaces

6.1 Microcontroller Pinout Strategy

The interface design maximizes the efficiency of the Arduino Uno's ATmega328P architecture. To support the Interrupt-Driven architecture, we have re-assigned pins to leverage Pin

Change Interrupts (PCINT) and direct Digital I/O.

Pin	Function	Description
D2, D4, D7, D8	Digital Input	Comparator Outputs (Interrupt Triggers)
D3, D5, D6, D9	Digital Output	Solenoid MOSFET Driver Gates
D0 (RX), D1 (TX)	UART	Serial Communication for Debugging
5V / GND	Power Rails	Logic Power for Sensors

Table 1: Updated System Pin Assignment Table

6.2 Sensor Interface (Comparator & Interrupts)

To achieve the microsecond-level reaction times required, we bypass the Arduino's slow internal ADC in favor of a hardware-based thresholding system.

- **Comparator Circuit:** Each phototransistor voltage is fed into the non-inverting input of an **LM339 Analog Comparator**. The inverting input is connected to a reference voltage (V_{ref}) set by a precision multi-turn potentiometer.
- **Zero-Latency Detection:** Instead of the CPU spending $100\mu s$ to digitize an analog signal (ADC conversion time), the comparator switches its output LOW immediately when the light level drops below the threshold. The propagation delay of the LM339 is approx. $1.3\mu s$, effectively instantaneous compared to the game speed.
- **Interrupt Trigger:** The comparator outputs are routed to Digital Pins configured for **Pin Change Interrupts (PCINT)**. This generates a hardware interrupt request (IRQ) exactly when the note arrives, waking the processor instantly to handle the event.

6.3 Digital Interface (Actuation)

The four solenoid drivers connect to Digital Pins 3, 5, 6, and 9.

- **Direct Digital Drive:** Unlike servos which require a continuous 50Hz PWM signal to hold position, the solenoids operate in a simple "One-Shot" mode. We configure these pins as standard Digital Outputs.
- **Timer-Controlled Pulse:** When the interrupt fires, we do not use `delay()` to hold the solenoid ON. Instead, we trigger a Hardware Timer to pull the pin HIGH, count exactly $20ms$, and then pull it LOW via an interrupt. This ensures the solenoid actuation duration is precise and does not block the main CPU loop.

7 Software

7.1 Control Logic Overview

The software will be developed in C++ using the Arduino IDE. The central architectural challenge is managing four independent game lanes simultaneously without blocking the processor. A standard linear approach using 'delay()' would render the robot "blind" to events in Lane 2 while it is pressing a button for Lane 1.

To resolve this, we will implement an **Interrupt-Driven Architecture** utilizing hardware timers. Rather than continuously polling the sensors in a loop (which consumes CPU cycles and introduces jitter), we will configure the analog comparators to trigger an **External Interrupt (ISR)** immediately upon detecting a note. Once triggered, the ISR will utilize a **Hardware Timer** to precisely schedule the solenoid actuation delay (Δt) and duration (t_{hold}), ensuring that the mechanical response is decoupled from the main processor loop.

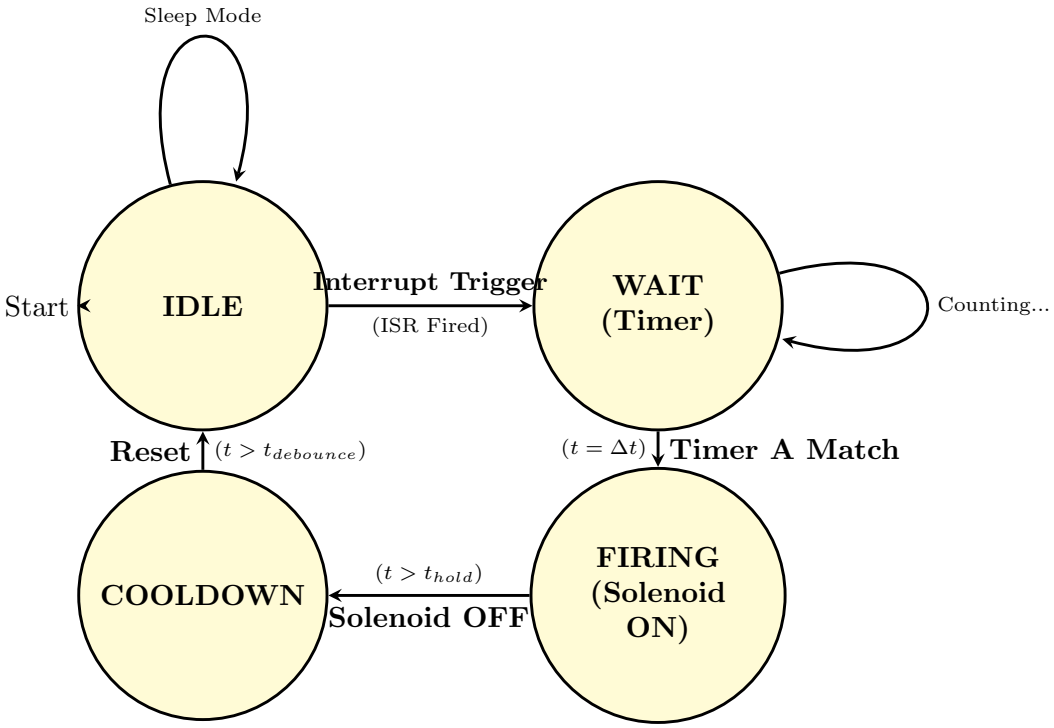

Figure 10: **Interrupt-Driven State Machine:** The logic flow for a single lane. Instead of polling, the system remains IDLE until a hardware interrupt triggers the Timer. The Solenoid is fired via a "One-Shot" timer pulse, ensuring precise duration without blocking the CPU.

7.2 Calibration and Startup Routine

Since the voltage thresholds are now set via hardware potentiometers (see Section 6.2), the software does not need to calculate V_{thresh}. Instead, the 'setup()' routine performs a **Safety Verification**:

1. **Logic Check:** The system reads all four Digital Input pins. It verifies that all comparators are outputting a Logic HIGH (IDLE state). If any pin is LOW (indicating a sensor is stuck or covered), the system enters an Error State and flashes the status LED.

2. **Solenoid Retract:** The system ensures all solenoid pins are written LOW to prevent accidental firing during startup, which could damage the screen or overheat the coils.

7.3 The Interrupt-Driven Logic

The core logic is split between the Interrupt Service Routine (ISR) (which handles the critical "Trigger" event) and the Main Loop (which handles timing and solenoid release).

Listing 1: Pseudocode for Interrupt-Driven Lane Logic

```
// Global Volatile Variables (Shared between ISR and Loop)
volatile bool triggered = false;
volatile unsigned long detectTime = 0;

// 1. THE INTERRUPT (Runs instantly when Comparator goes LOW)
void ISR_Lane1() {
  if (state == IDLE) {
    detectTime = micros(); // Capture exact microsecond
    triggered = true;      // Flag main loop to handle it
    state = WAIT_LATENCY;
  }
}

// 2. THE MAIN LOOP (Non-Blocking)
void updateLane() {
  unsigned long currentMicros = micros();

  switch (state) {
    case WAIT_LATENCY:
      // Wait for 'Look Ahead' delay (delta_t)
      if (currentMicros - detectTime >= TRAVEL_DELAY) {
```

```
        digitalWrite(SOLENOID_PIN, HIGH); // FIRE!
        strikeTime = currentMicros;
        state = FIRING;
      }
      break;

    case FIRING:
      // Hold Solenoid ON for 20ms to ensure touch registration
      if (currentMicros - strikeTime >= HOLD_DURATION) {
        digitalWrite(SOLENOID_PIN, LOW); // RELEASE
        cooldownStart = currentMicros;
        state = COOLDOWN;
      }
      break;

    case COOLDOWN:
      // Ignore interrupts for 50ms to prevent double-hits
      if (currentMicros - cooldownStart >= DEBOUNCE) {
        state = IDLE;
        triggered = false; // Reset flag
      }
      break;
  }
}
```

7.4 Libraries and Timing Architecture

We explicitly discard the `<Servo.h>` library, as solenoids are binary devices that do not require a 50Hz PWM carrier signal.

- **Microsecond Precision:** Instead of standard `millis()`, we utilize the `micros()` counter and hardware interrupts. This allows us to track the note's position with a resolution of $4\mu s$ (on the Arduino Uno), which is critical when the scroll speed exceeds 50cm/s.

- **Port Manipulation (Optimization):** To further reduce latency, we may bypass the standard `digitalWrite()` function (which takes $\approx 5\mu s$) and write directly to the ATmega328P's **PORTD registers**. This allows us to trigger the solenoid driver in a single clock cycle ($62.5ns$).

8 Testing Plan

8.1 Phase 1: Sub-System Verification

Testing will proceed in a modular fashion to verify the new hardware architecture.

- **Comparator Tuning & Digital Logic Test:** Since we are using hardware comparators, we cannot plot "raw analog values." Instead, we will monitor the Digital Input pins via the Serial Monitor.
  - *Procedure:* We will pass a black test card under the sensor while adjusting the multi-turn potentiometer on the LM339 comparator circuit.
  - *Success Criteria:* The Serial Monitor must report a clean, instant transition from Logic HIGH (White) to Logic LOW (Black) without "flickering" (bouncing) at the transition edge.

- **Solenoid Driver & Thermal Test:** We will run a "Rapid Fire" routine that toggles all four solenoids at 5Hz (standard gameplay speed) for 60 seconds.
  - *Objective:* Verify that the 6V Power Rail does not sag below 5.5V during the current spikes. Crucially, we will use an IR thermometer to verify that the MOSFET

drivers and Solenoid Coils remain within safe thermal limits ($< 50°C$).

8.2 Phase 2: Integration and Latency Tuning

Once hardware is verified, we will close the loop using a controlled video test.

8.3 Phase 2: Integration and Latency Tuning

Once hardware is verified, we will close the loop using a controlled video test.

- **Static Latency Calibration:** We will set a hard-coded delay (Δt) based on our theoretical calculation (Equation 2). We will record the robot playing a standard level using a smartphone camera at 240 fps (Slow Motion). By counting frames between the target line arrival and the solenoid impact, we can calculate the exact timing error (δt) and adjust the 'travelDelay' variable to compensate for electromechanical inductance.
- **Variable Velocity Verification (The "Simulator" Test):** To satisfy the requirement of testing at different speeds scientifically, we cannot rely on the random pacing of the commercial game. We will place the robot over a **high-refresh-rate monitor running a custom scrolling simulator** (coded in Python/Processing). This allows us to "program the screen" to validate performance at distinct velocity tiers:
  - **Tier 1 (Cruising):** 1 notes/sec (Standard game start speed).
  - **Tier 2 (Human Speed):** 2 notes/sec (Mid-game speed).
  - **Tier 3 (Stress):** 5+ notes/sec (Approaching mechanical limits).
- **Acceleration Tracking:** Since *Piano Tiles* speeds up continuously, we will program the simulator to ramp the scroll velocity ($v(t) = v_0 + at$) to verify that the robot's "Look Ahead" logic holds true even as Δt shrinks dynamically.

- *Prediction:* With the switch to Solenoids, we eliminate the 60ms servo travel time. We expect the thermal and inductive limits of the solenoid coils (approx. 15ms extend/retract cycle) to raise our ceiling to $\approx 30 - 40$ notes per second per lane.

9 Safety

9.1 Electrical Safety

While the project operates at Extra-Low Voltage (< 12V), the use of high-current inductive loads introduces specific risks.

- **Short Circuit Prevention:** The high current capability of the solenoid power rail means a VCC-to-GND short could rapidly heat wiring. We will mitigate this by setting the Current Limit (CC) on the power supply to 2.0A and insulating all soldered joints.
- **Inductive Kickback Protection:** Solenoids generate high-voltage spikes (Back-EMF) when de-energized. It could help to install flyback diodes in parallel with each coil to clamp these spikes, preventing damage to the MOSFETs or the Arduino.
- **Polarity Check:** We will color-code all power leads (Red for VCC, Black for GND) to prevent reverse-polarity connections, which would destroy the MOSFET driver circuits.

9.2 Mechanical and Device Safety

- **Pinch Points:** The solenoid plungers oscillate rapidly ($f > 5$Hz). While the linear force is low ($\approx 2 - 5$N), it is sufficient to pinch skin. We will route all cabling through the hollow sections of the bridge, keeping wires clear of the moving plungers.
- **Device Protection (Smartphone):** The most significant financial risk is damage to the smartphone/tablet screen.

- *Soft Impact:* The tips of the plungers are cushioned with compressible conductive foam, ensuring that even a maximum-force impact will not crack the glass.
- *Thermal Watchdog (Software Limit):* Unlike servos, solenoids heat up rapidly if left energized. We will implement a "Max On-Time" safety check in the ISR: if a solenoid remains active for $> 100ms$ (due to a software crash), the system will force the pin LOW to prevent coil burnout.
- *Passive Emergency Stop:* The solenoids feature an internal return spring. In the event of power loss, USB disconnection, or code failure, the plungers automatically retract to the safe "Hover" position, physically decoupling them from the screen.

10 Parts, Reusability, and Shopping List

10.1 Components on Hand (University Inventory)

We will leverage existing laboratory stock to minimize costs and waste.

- **Arduino Uno R3:** The primary microcontroller platform.
- **Passive Components:** We will utilize lab stock for all 10kΩ pull-up resistors, 220Ω LED current-limiting resistors, and hookup wire.
- **Structural Material:** The bridge chassis will be fabricated entirely from scrap acrylic and plywood found in the physics workshop bins.
- **Phototransistors:** We will be using in-house PT5529/L2-F Phototransistors.
- **Benchtop Power Supply:** Variable DC power supply available in the lab to drive the high-current solenoid rail.

10.2 Components to Purchase (Bill of Materials)

We have identified specific components that require purchasing to ensure high reliability and future reusability for the department.

Table 2: Project Bill of Materials

Component	**Qty**	**Cost**	**Technical Justification & Reusability**
Linear Solenoid (5V/6V)	2 (We have 2)	≈$20	**Justification:** Replaces servos to eliminate mechanical latency ($T_{mech} < 15ms$). **Reusability:** High-speed linear actuators useful for future physics demos. Amazon (B0B4DFJTHX)
PT5529B/L2-F Phototransistors	4	$0	**Justification:** Chosen over LDRs for microsecond-scale rise times required for high-speed game detection. (In-house).
FSR 400 Force Sensors	4	$25	**Justification:** Required to determine moment of contact and location. `https://www.dfrobot.com/product-614.html`
Continued on next page			

Table 2 – continued from previous page

Component	Qty	Cost	Technical Justification & Reusability
Conductive Foam	1	$16	**Justification:** Provides variable resistance contact for capacitive screens. https://a.co/d/dVHOCnV
Total		**≈$61**	

11 Schedule and Milestones

11.1 Overview

We have allocated approximately 12 hours per week for this project, split between individual component testing and joint integration sessions in the physics lab. The schedule is designed to front-load the mechanical construction to allow maximum time for software tuning (latency compensation) in the final weeks, as we anticipate the timing calibration to be the most time-consuming task.

11.2 Weekly Breakdown

Table 3: Project Execution Schedule

Week / Dates	**Phase & Activities**	**Key Dates & Milestones**
Week 1 (Feb 2 – Feb 8)	**Mechanical Construction** Fabricate acrylic/wood "Bridge." Mount sensors and solenoids. Align assembly with phone screen.	**Feb 3: PROPOSAL DUE.** **Feb 4:** Proposal Feedback. *Milestone:* Bridge assembled.
Week 2 (Feb 9 – Feb 15)	**The "One Lane" Test** Code single-lane logic (Δt calibration). Verify "Look Ahead" math.	**Feb 10: MIDTERM.** **Feb 11:** Lab 4 Report / Updated Proposal Due.
Week 3 (Feb 16 – Feb 22)	**Full Integration** Scale code to 4 lanes. Implement Start-up Calibration routine. Finalize cable management.	*Milestone:* Full 4-lane autonomous operation (beta).
Week 4 (Feb 23 – Mar 1)	**Optimization & Check-in** Stress testing at high speeds. Mechanical vibration analysis. Fine-tune thresholds.	**Feb 25: MID-PROJECT CHECK-IN.** *Milestone:* Working Demo.
Week 5 (Mar 2 – Mar 8)	**Advanced Features** General trouble-shooting and a week to breath. Try to implement "Reach Goals" (e.g., Automatic Latency Calibration or Sustain Logic). Refine FSM logic.	*Milestone:* System feature complete.
Continued on next page		

Table 3 – continued from previous page

Week / Dates	Phase & Activities	Key Dates & Milestones
Week 6 (Mar 9 – Mar 15)	**Documentation & Polish** Film demonstration video. Write final project report. Final code cleanup and commenting.	*Milestone:* Video filmed, Report drafted.
Week 7 (Mar 16 – Mar 22)	**Final Presentation** Final system check. Charge power supply/batteries. Presentation preparation.	**Mar 17: FINAL DEMO** (8:00am – 11:00am).

12 Mid-Project Deliverables (Due Feb 25)

For the mid-project check-in, we will demonstrate the core functionality of the hardware and the viability of the sensing logic. Given the complexity of integrating four parallel analog channels, we will focus our live demonstration on a single lane to prove the physics, while showing mechanical readiness for the full system. Specifically, we will deliver:

- **Mechanical Assembly:** The complete "Bridge" structure will be built, with all 4 solenoids and 4 sensors physically mounted, wired, and aligned over the phone.
- **Sensing Integration:** We will visualize the Digital Logic stream via the Serial Plotter. Since the comparators output a binary 0 or 1, we will demonstrate clean, crisp transitions (no flickering) when a tile passes, verifying that the hardware sensitivity knobs are correctly tuned.
- **Actuation Demo:** We will demonstrate at least **one single finger** successfully tapping a tile in response to a sensor input (The "One Lane" Test). This proves the closed-loop latency management is functioning.

Note: By prioritizing the perfection of a single lane, we establish a stable standard for timing and sensitivity. Once this logic is verified, scaling the code to the remaining three lanes is a matter of software iteration rather than physical reconstruction.

13 Comprehensive Risk Mitigation and Scalability

13.1 Expansion Scenarios (The "Reach" Goals)

If core mechanics are in place and we're on pace to have extra time, we will implement advanced signal processing features. These features involving a deeper level of processing the "keys" of the "piano" to perform other outcomes.

13.1.1 Automatic Latency Calibration (ALC)

Currently, the user must hard-code the `travelDelay` variable. An expansion goal is to automate this process.

- **Concept:** For each and every note, the robot will record two different positions in a set time interval.
- **Algorithm:** The Arduino will determine the speed of each and every note, then determine the timing it should take to hit the note at a specific distance.

13.1.2 Handling "Long Notes" (Sustain Logic)

The Physics Problem: Long notes appear as a continuous black bar. A simple "edge detection" trigger would tap once at the start and then release, resulting in a failed note.

The Solution: We will modify the sensor logic to differentiate between a "Transient Pulse" (short tile) and a "Steady State" (long tile). This requires rewriting the non-blocking state machine to include a `SUSTAINING` state.

Listing 2: Conceptual Logic for Sustain Notes

```
if (digitalRead(COMPARATOR_PIN) == LOW) { // Note Detected
    digitalWrite(SOLENOID_PIN, HIGH);     // Hold Down

    while (digitalRead(COMPARATOR_PIN) == LOW) {
        // Busy wait or state check:
        // Keep holding while the black bar is passing
    }

    digitalWrite(SOLENOID_PIN, LOW);      // Release
}
```

13.2 De-Scope Plans (Failure Management)

Project failure often stems from the accumulation of small latencies, mechanical inertia, code execution time, and sensor rise time. We have defined specific "fallback" configurations if the full high-speed game proves impossible.

13.2.1 Fallback A: Optical Isolation (Crosstalk Elimination)

- **The Failure Mode:** On small phone screens, the lanes are close together. The cone of sensitivity of Sensor A might overlap with Lane B, causing false positives.
- **The Fix:** We will mechanically restrict the system to use only Lane 1 and Lane 3, leaving Lanes 2 and 4 empty.
- **Impact:** This physically doubles the separation distance between active sensors, effectively eliminating optical crosstalk without requiring complex software filtering.

13.2.2 Fallback B: Reduced Game Speed (Kinematic Margin)

- **The Failure Mode:** If the solenoid inductance prevents it from retracting fast enough for the highest difficulty levels (e.g., > 12 notes/sec).
- **The Fix:** We will restrict the demonstration to "Easy" or "Medium" difficulty settings where note density is lower.
- **Impact:** This preserves the full 4-lane architecture and physics model but acknowledges the thermal/magnetic limits of the specific solenoids chosen.